

TP10 : Partage du processeur / Commutation de tâches

A. Objectifs

Ce TP porte sur les mécanismes permettant à un processeur de fonctionner en mode multi-tâches, c'est à dire d'exécuter plusieurs programmes utilisateurs simultanément. On utilise pour cela la technique de multiplexage temporel, où chaque processeur travaille successivement pour différents programmes durant des tranches de temps de longueur fixe. L'objectif du TP est d'identifier les problèmes liés à la création, à la sauvegarde, et à la restauration de tâches logicielles, et d'analyser en détail le mécanisme de commutation de tâches.

B. Architecture matérielle

tp9_topcell

Question 1

Pour cette questions il faut rajouter les signaux d'interruption à l'icu de la maniere suivante :

```
icu.p_irq_in[0]          (signal_irq_ioc)
```

Il faut prendre en compte les ports non utilisés dans les cas ou: $p \geq nprocs$ ou $t > 4$.

C. Contexte d'exécution d'une tâche

Question C1: Rappelez le principe du pseudo-parallélisme par multiplexage temporel sur un seul processeur. Quel est l'événement qui provoque la commutation de tâches ? Utilisez le chronogramme ci-dessous pour expliquer les différentes étapes du mécanisme de commutation de tâches. Indication: Vous pouvez suivre les appels de fonctions à partir de l'exécution de l'ISR `_isr_switch` (dans le fichier `giet_2011/sys/irq_handler.c`).

le pseudo-parallélisme par multiplexage temporel sur un seul processeur consiste a gerer la fréquence d'exécution des taches afin de donner une illusion de parallelisme à l'utilisateur.

l'événement qui provoque la commutation de tâches dans notre architecture est la fin du quantum et la levée d'une interruption TIMER.

tp9_chronogramme

- 1- Exécution de la tâche T0
- 2- Interruption TIMER
- 3- Traitement de l'interrutpition ,élection d'une tâche , déchargement de la tâche sortante et chargement de la tâche entrante

- 4- Sortie de l'ISR, de la fonction de commutation et du GIET
- 5- Exécution de la tâche T1
- 6- Interruption TIMER
- 7- Traitement de l'interruption, exécution de la fonction de commutation
- 8- Sortie de l'ISR, de la fonction de commutation et du GIET
- 9- Exécution de la tâche T0

Question C2: Où est sauvegardé le contexte d'une tâche ? Pourquoi n'est-il pas nécessaire de sauvegarder les valeurs stockées dans la pile lorsqu'on effectue un changement de contexte ?

Le contexte d'une tâche est sauvegardé dans un tableau `_task_context_array[NB_PROCS * NB_MAXTASKS * TASK_CTXT_SIZE]`.
Il n'est pas nécessaire de sauvegarder les valeurs stockées dans la pile car chaque tâche a une pile qui lui est propre.

Question C3: Le GIET utilise une politique d'assignation statique des tâches aux processeurs. Cela signifie qu'il n'y a pas de migration des tâches, et qu'une tâche particulière sera toujours exécutée sur le même processeur. Comment le GIET mémorise-t-il le placement des tâches (identifiées par leur index global `task_id`) sur les processeurs ?

Le GIET mémorise le placement des tâches grâce au tableau `_task_number_array[NB_PROCS]`

Question C4: Pour chaque processeur, c'est le système d'exploitation qui contrôle l'ordonnancement entre les tâches assignées à ce processeur. Comment le GIET mémorise-t-il quelle tâche est en cours d'exécution sur chacun des processeurs à un instant donné ?

Afin de mémoriser quelle tâche est en cours d'exécution sur chacun des processeurs à un instant donné, `_current_task_array[NB_PROCS]`

Question C5: Parmi les 32 registres généraux du processeur MIPS32, quels sont ceux qu'il n'est pas nécessaire de sauvegarder lors d'un changement de contexte ? Parmi les registres système, justifiez pourquoi il est indispensable de sauvegarder SR, EPC, CR ?

Il n'est pas nécessaire de sauvegarder les registres \$0, \$26 et \$27 car 0 est une constante et \$26 et \$27 ne sont pas utilisés par la tâche car ils sont réservés au noyau.

SR: Pour restaurer exactement les conditions d'exécution de la tâche lors de sa reprise.
EPC: Afin de savoir où reprendre l'exécution de la tâche
CR: Pour que la cause de l'interruption ne soit pas écrasée.

Question C6: Quels sont les arguments de cette fonction ? Comment la fonction récupère-t-elle ses arguments ? Quelle est sa valeur de retour ?

La fonction reçoit deux arguments représentant l'adresse du contexte de la tâche. Ces arguments sont récupérés depuis le registre \$4 et mis dans le registre \$27.

La valeur de retour de la fonction est l'adresse du contexte de la tâche suivante.

Question C7: La fonction `_task_switch` est découpée en deux parties. Que cherche-t-on à faire dans chacune de ces 2 parties ? Quels registres cette fonction a-t-elle le droit de modifier dans la phase de sauvegarde du contexte de la tâche sortante ?

Partie 1: Sauvegarder le contexte de la tâche actuelle.

Partie 2: Restaurer le contexte de la tâche qui va s'exécuter.

Question C8: Pourquoi la fonction `_task_switch` qui réalise la commutation de contexte (et qu'on trouve dans tous les systèmes d'exploitation multi-tâches) est elle toujours écrite en assembleur ?

La fonction `_task_switch` est toujours écrite en assembleur car la manipulation des registres est plus simple.

Question C9: A quelle adresse se branche la fonction `_task_switch` quand elle retourne à la fonction appelante ? Pourquoi faut-il initialiser la case contenant normalement la sauvegarde du registre \$31, dans tableau de contexte d'une tâche (avant qu'elle ne soit exécutée pour la première fois) ? Quelle valeur doit contenir cette case 31 du tableau de contexte ?

La fonction `_task_switch` se branche à l'adresse contenu dans la case `ctx[31]` de la nouvelle tâche.

L'initialisation de la case contenant la sauvegarde du registre \$31 permet de faire sauter début de la fonction de la tâche.

Question C10: La fonction `_task_switch` est appelée par la fonction `_ctx_switch`. Cette fonction `_ctx_switch` est chargée de choisir la tâche entrante, ce qui suppose la définition d'une politique d'ordonnancement entre les tâches. Quel est la politique d'ordonnancement implémentée par la fonction `_ctx_switch` ?

La politique d'ordonnancement implémentée par la fonction `_ctx_switch` est le round-robin